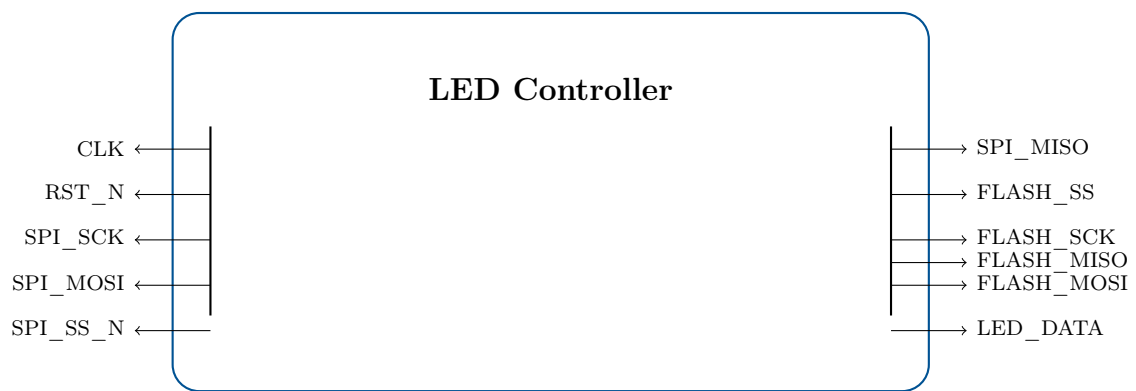


LED Controller Module

Programmable SK6812RGBW LED Driver
with Microprogram Architecture



Technical Datasheet

Document Version 1.0
January 25, 2026

Contents

1	Overview	3
1.1	Key Features	3
1.2	Applications	3
2	Pin Description	3
2.1	Tiny Tapeout Pinout	4
3	SPI Register Interface	4
3.1	SPI Protocol	4
3.2	Register Map	5
3.3	Effect Register Values	5
3.4	Color Format	5
4	Microprogram Architecture	5
4.1	CPU Specifications	6
4.2	Instruction Set	6
4.2.1	Load Instructions	6
4.2.2	Store Instructions	6
4.2.3	Compare Instructions	6
4.2.4	Increment/Decrement Instructions	7
4.2.5	Branch and Jump Instructions	7
4.2.6	Special Instructions	7
4.3	Memory-Mapped Registers	7
5	SK6812RGBW Protocol	8
5.1	Timing Specifications	8
5.2	Implementation	8
6	SPI Flash Interface	8
6.1	XIP (Execute-In-Place)	8
6.1.1	Cache Implementation (Future Enhancement)	8
6.2	Flash Specifications	9
6.3	Address Width Configuration	9
6.4	Read Sequence	9
7	Timer System	9
8	Electrical Characteristics	10
8.1	Recommended Operating Conditions	10
8.2	Clock Divider Calculation	10
9	Reset Behavior	10
10	Application Notes	11
10.1	Connecting SK6812RGBW LEDs	11
10.2	Programming via SPI	11
10.3	Writing Custom Programs	11
11	Revision History	11

1 Overview

The LED Controller is a fully programmable SK6812RGBW LED strip driver designed for the Tiny Tapeout platform. It features a custom 8-bit microprocessor architecture that reads programs from external SPI flash memory, enabling complex LED animations without hardware modifications.

1.1 Key Features

- SK6812RGBW LED protocol support (32-bit GRBW format)
- Configurable LED count (up to 255 LEDs)
- Up to 3 configurable colors (96 bits total)
- Built-in LED effects: None, Chase, Pulse
- Custom microprogram execution from SPI flash
- SPI slave interface for runtime configuration
- Configurable clock divider for various system frequencies
- Chip ID: 0x6925

1.2 Applications

- Decorative LED lighting
- Status indicators
- Custom animations and displays
- Wearable electronics
- Interactive art installations

2 Pin Description

Table 1: Pin Description

Pin Name	Direction	Active	Description
<i>System Signals</i>			
i_clk	Input	–	System clock (default 50MHz)
i_reset_n	Input	Low	Active-low asynchronous reset
<i>SPI Slave Interface (Configuration)</i>			
i_spi_sck	Input	–	SPI clock
i_spi_mosi	Input	–	SPI Master Out, Slave In
i_spi_ss_n	Input	Low	SPI slave select (active low)
o_spi_miso	Output	–	SPI Master In, Slave Out
<i>SPI Flash Interface (Program Memory)</i>			
o_flash_spi_ss_n	Output	Low	Flash chip select
o_flash_spi_mosi	Output	–	Flash data output
o_flash_spi_sck	Output	–	Flash SPI clock
i_flash_spi_miso	Input	–	Flash data input
<i>LED Output</i>			
o_data	Output	–	SK6812RGBW serial data output

2.1 Tiny Tapeout Pinout

The following tables show the mapping of signals to Tiny Tapeout pins.

Table 2: Dedicated Inputs (ui_in)

Pin	Signal	Description
ui[0]	SPI_SCK	SPI slave clock
ui[1]	SPI_SS_N	SPI slave select (active low)
ui[2]	SPI_MOSI	SPI slave data in
ui[3]	FLASH_MISO	SPI flash data out
ui[4–7]	–	Unused

Table 3: Dedicated Outputs (uo_out)

Pin	Signal	Description
uo[0]	FLASH_SS_N	SPI flash chip select (active low)
uo[1]	FLASH_MOSI	SPI flash data in
uo[2]	FLASH_SCK	SPI flash clock
uo[3]	LED_DATA	SK6812RGBW serial data output
uo[4–7]	–	Unused

Table 4: Bidirectional IOs (uio)

Pin	Signal	Direction	Description
uio[0]	SPI_MISO	Output	SPI slave data out (active when SS_N low)
uio[1–7]	–	Input	Unused

3 SPI Register Interface

The LED Controller exposes a register-based interface over SPI for runtime configuration. The SPI interface operates in Mode 0 (CPOL=0, CPHA=0).

3.1 SPI Protocol

- **Read:** Send register address (bit 7 = 0), receive data on next byte
- **Write:** Send register address with bit 7 = 1, then send data byte
- Auto-increment addressing for consecutive reads/writes

3.2 Register Map

Table 5: Register Map

Addr	Name	R/W	Default	Description
0x01	EFFECT	R/W	0x03	Active effect (see below)
0x02	CHIP_ID_H	R	0x69	Chip ID high byte
0x03	CHIP_ID_L	R	0x25	Chip ID low byte
0x04–0x07	COLOR0	R/W	0x00FF0000	Color 0 (GRBW format)
0x08–0x0B	COLOR1	R/W	0xFF000000	Color 1 (GRBW format)
0x0C–0x0F	COLOR2	R/W	0x0000FF00	Color 2 (GRBW format)
0x10	NUM_LEDS	R/W	30	Number of LEDs (1–255)
0x11	NUM_COLORS	R/W	3	Number of active colors (1–3)
0x12	CLK_DIV	R/W	5	Clock divider for 100ns ticks
0x13	FLASH_24_BIT	R/W	1	Flash address width (1=24-bit, 0=16-bit)
0x14	CPU_RESET	W	–	Write any value to reset CPU and flash state

3.3 Effect Register Values

Table 6: Effect Types

Value	Effect	Description
0	NONE	Static display, all LEDs same color, cycles through colors
1	CHASE	Chase animation, colors shift at 2Hz
2	PULSE	Breathing/pulsing effect (~2.5 second cycle)
3	CUSTOM	Execute microprogram from SPI flash

3.4 Color Format

Colors are stored in 32-bit GRBW format (Green-Red-Blue-White):

Bits 31–24	Bits 23–16	Bits 15–8	Bits 7–0
Green	Red	Blue	White

4 Microprogram Architecture

The LED Controller includes a custom 8-bit microprocessor for executing LED animation programs stored in SPI flash memory.

4.1 CPU Specifications

Table 7: CPU Specifications

Parameter	Value
Data Width	8-bit
Instruction Width	16-bit (8-bit opcode + 8-bit operand)
Program Memory	256 bytes (from SPI flash)
Scratch Memory	16 bytes (addresses 0x10–0x1F)
Registers	X (8-bit), Y (8-bit), PC (8-bit)
Status Flags	Zero (Z), Negative (N)

4.2 Instruction Set

4.2.1 Load Instructions

Table 8: Load Instructions

Opcode	Binary	Mnemonic	Description
0x01	0001	ldx #imm	Load immediate value into X
0x02	0010	ldy #imm	Load immediate value into Y
0x0A	1010	ldx \$addr	Load from address into X
0x0B	1011	ldy \$addr	Load from address into Y

4.2.2 Store Instructions

Table 9: Store Instructions

Opcode	Binary	Mnemonic	Description
0x0E	1110	sty \$addr	Store Y to address
0x0F	1111	stx \$addr	Store X to address

4.2.3 Compare Instructions

Table 10: Compare Instructions

Opcode	Binary	Mnemonic	Description
0x03	0011	cpx #imm	Compare X with immediate
0x04	0100	cpy #imm	Compare Y with immediate
0x0C	1100	cpx \$addr	Compare X with address value
0x0D	1101	cpy \$addr	Compare Y with address value

4.2.4 Increment/Decrement Instructions

Table 11: Increment/Decrement Instructions

Opcode	Binary	Mnemonic	Description
0x05	0101	iny	Increment Y register
0x06	0110	inx	Increment X register
0x13	10011	dey	Decrement Y register
0x14	10100	dex	Decrement X register

4.2.5 Branch and Jump Instructions

Table 12: Branch and Jump Instructions

Opcode	Binary	Mnemonic	Description
0x07	0111	bne offset	Branch if not equal (Z=0)
0x08	1000	beq offset	Branch if equal (Z=1)
0x10	10000	jmp addr	Unconditional jump (absolute)
0x11	10001	bmi offset	Branch if minus (N=1)
0x12	10010	bpl offset	Branch if plus (N=0)

Branch Offset Calculation: Branch target = PC + 2 + offset. The offset is a signed 8-bit value (−128 to +127), allowing both forward and backward branches. The +2 accounts for the 16-bit instruction width.

4.2.6 Special Instructions

Table 13: Special Instructions

Opcode	Binary	Mnemonic	Description
0x00	0000	nop	No operation
0x09	1001	stall #n	Delay $n \times 10\text{ms}$

4.3 Memory-Mapped Registers

The CPU can access special hardware registers through memory addresses:

Table 14: Memory-Mapped Hardware Registers

Address	Name	Description
\$0	led_count	Number of LEDs (read-only)
\$1	color_count	Number of colors configured (read-only)
\$2	write_led	Write LED pixel (write: color index 0–2, 3=off)
\$10–\$1F	scratch	16 bytes of scratch memory

5 SK6812RGBW Protocol

5.1 Timing Specifications

The SK6812RGBW LEDs use a single-wire protocol with the following timing:

Table 15: SK6812RGBW Timing Parameters

Parameter	Symbol	Typical	Unit
Logic 0 High Time	T0H	300	ns
Logic 0 Low Time	T0L	900	ns
Logic 1 High Time	T1H	600	ns
Logic 1 Low Time	T1L	600	ns
Reset Time	TRESET	80	μ s

5.2 Implementation

The `sk6812rgbw` module generates the waveform using a configurable clock divider:

- **Tick period:** $100\text{ns} \times \text{clock_divider}$
- **Logic 0:** 3 high ticks, 9 low ticks (300ns high, 900ns low)
- **Logic 1:** 6 high ticks, 6 low ticks (600ns high, 600ns low)
- **Reset:** 800 ticks ($80\mu\text{s}$)

6 SPI Flash Interface

The SPI flash interface provides Execute-In-Place (XIP) functionality for the microprocessor, reading program instructions directly from external SPI flash memory.

6.1 XIP (Execute-In-Place)

The LED Controller implements XIP to execute programs directly from SPI flash without requiring internal program memory. This approach enables:

- Unlimited program size (limited only by flash capacity)
- Easy program updates by reprogramming the flash
- Minimal on-chip memory requirements

Important: XIP mode is **enabled by default**. On power-up, the controller immediately begins executing code from SPI flash address 0x0000. Ensure valid program code is present in flash before powering on the device.

6.1.1 Cache Implementation (Future Enhancement)

A cache line implementation exists in the design (`spi_flash_cache.sv`) with configurable cache line size and number of cache lines. However, due to area constraints on the Tiny Tapeout 2x2 tile, the cache module is not instantiated in the current build. This may be included in future larger tile implementations.

The cache parameters (when enabled) are:

- Cache line size: 32–64 bytes
- Number of cache lines: 2
- Total cache: 64–128 bytes (32–64 instructions)

6.2 Flash Specifications

Table 16: SPI Flash Interface Parameters

Parameter	Value
SPI Mode	Mode 0 (CPOL=0, CPHA=0)
SPI Clock Divider	2 (fixed, relative to system clock)
Maximum SPI Clock	25 MHz (at 50 MHz system clock)
Read Command	0x03 (Standard Read)
Address Width	24-bit (default) or 16-bit
Data Width	8-bit

Note: The SPI flash clock operates at half the system clock frequency. At the default 50 MHz system clock, the flash SPI clock is 25 MHz. This ensures compatibility with most standard SPI flash devices which typically support 25–50 MHz operation.

6.3 Address Width Configuration

The flash interface supports both 16-bit and 24-bit addressing modes:

Table 17: Flash Address Width Options

Mode	Register Value	Addressable Range
24-bit (default)	FLASH_24_BIT = 1	16 MB
16-bit	FLASH_24_BIT = 0	64 KB

24-bit addressing is enabled by default to support larger flash devices. When using small flash chips (64KB or less), 16-bit mode can be enabled for slightly faster instruction fetch.

6.4 Read Sequence

1. Assert chip select (CS_N = 0)
2. Send read command (0x03)
3. Send address (24-bit or 16-bit, MSB first)
4. Receive data bytes (continuous read while CS_N low)
5. Deassert chip select (CS_N = 1)

7 Timer System

The LED Controller includes a hierarchical timer system:

Table 18: Timer Configuration

Timer	Frequency	Period	Usage
Base Timer	100 Hz	10 ms	CPU stall timing
10 Hz Timer	10 Hz	100 ms	Pulse effect brightness steps
2 Hz Timer	2 Hz	500 ms	Chase effect
2s Timer	0.5 Hz	2 s	Long delays

The base timer uses 1,000,000 clock ticks with the configured clock divider to achieve 10Hz at 50MHz system clock.

8 Electrical Characteristics

8.1 Recommended Operating Conditions

Table 19: Recommended Operating Conditions

Parameter	Symbol	Min	Typ	Max
System Clock Frequency	f_{CLK}	10	50	100 MHz
Clock Divider (for 50MHz)	CLK_DIV	–	5	–
Supply Voltage	VDD	3.0	3.3	3.6 V

8.2 Clock Divider Calculation

To achieve correct SK6812RGBW timing, set the clock divider based on system frequency:

$$CLK_DIV = \frac{f_{CLK} \text{ (MHz)}}{10}$$

For example:

- 20 MHz: CLK_DIV = 2
- 30 MHz: CLK_DIV = 3
- 40 MHz: CLK_DIV = 4
- 50 MHz: CLK_DIV = 5
- 100 MHz: CLK_DIV = 10

9 Reset Behavior

On reset, the controller initializes to the following state:

Table 20: Default Reset Values

Parameter	Default Value
Effect Mode	CUSTOM (0x03) – XIP enabled
Number of LEDs	30
Number of Colors	3
Clock Divider	5
Flash Address Width	24-bit
Color 0	0x00FF0000 (Green)
Color 1	0xFF000000 (Red)
Color 2	0x0000FF00 (Blue)
CPU Program Counter	0x00

The reset synchronizer uses a 2-stage flip-flop chain to prevent metastability issues with asynchronous reset de-assertion.

10 Application Notes

10.1 Connecting SK6812RGBW LEDs

Connect `o_data` to the DIN pin of the first LED in the strip.

10.2 Programming via SPI

Example SPI write sequence to change effect to CHASE:

```
1. Assert SS_N (low)
2. Send: 0x81 (Write to register 0x01)
3. Send: 0x01 (EFFECT_CHASE)
4. Deassert SS_N (high)
```

Listing 1: SPI Write Example

Example SPI read sequence to read chip ID:

```
1. Assert SS_N (low)
2. Send: 0x02 (Read register 0x02)
3. Receive: 0x69 (CHIP_ID_H)
4. Receive: 0x25 (CHIP_ID_L, auto-increment)
5. Deassert SS_N (high)
```

Listing 2: SPI Read Example

10.3 Writing Custom Programs

1. Write assembly code using the instruction set
2. Assemble to 16-bit binary (opcode:operand pairs)
3. Program into SPI flash starting at address 0x0000
4. Set effect mode to CUSTOM (0x03)

11 Revision History

Table 21: Document Revision History

Version	Date	Description
1.0	January 25, 2026	Initial release